About

This lab is about practicing variables and expressions using Python.

Learning Objectives

1. Learn to use the `input()` and `print()` functions
2. Practice using variables to store information
3. Practice basic arithmetic expressions

Estimated Size

3 small programs/files. While working on each question, you can submit a file individually to Gradescope to verify your work. In the end, you must submit all 3 files together to get full credit.

Tips - Preamble

The `input()` and `print()` functions will be integral to this assignment.

`input()` prompts the user for input, and returns what the user has typed as a string. The `input()` function can optionally take a string as the prompt text/message. So the command

```
s = input("Enter Input: ")
```

would print out **`Enter Input:`** to the terminal, await the input, and then store the text the user entered as a string in variable `s`.

Since `input()` always returns a string, if you want it converted to a number to use in arithmetic expressions, you can use **type casting** to convert it into an `int` or `float`.

`print()` can take in data or an expression of any type as a parameter, and print it to the terminal. So

```
print("Received: " + s)
```

(assuming `s` is a string) would print `Received:` followed by the content of `s`. It can also take multiple parameters and print them on the same line, separated by a space character in between. For example

```
print(1, "abc", s)
```

will print the number `1`, the string `"abc"`, and the content of variable `s` on the same line, separated by the space character.

`print()` always ends with a new line. So

```
print(1)
print("abc")
print(s)
```

will print the three items on separate lines. If you want to print a number `n` following some text:

```
print("n: " + str(n))
```

will work (note the **type casting**: the integer `n` must be converted to a string in order to be concatenated with another string).

0. Setup

In your Python files, you must include author information. On the first line, add a **`# Author`** comment line, where you mention your full name. Similarly, include your email and Spire ID on the **`# Email`** and **`# Spire ID`** comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author   : John Snow
# Email    : jsnow@umass.edu
# Spire ID : 12345678
```

1. A starting example `echo.py` (1 point)

The first program is provided to you as a starting example. It's already implemented. All you need to do is download the file, open it in VSCode, and change the first three lines of comments to include your information. You do NOT need to change the program code. This example reads in a user input, stores it in a variable, and prints it back to the terminal. After you have edited the comments, save it. You can then run it, try some different inputs, and verify the outputs are correct.

File download: echo.py

2. Implement `ana.py` (4 points)

This is a new program you must create from scratch. In VSCode, create a new file and name it `ana.py`. Note that the file name is **case-sensitive**: all letters must be lower-case. Naming it `Ana.py` or `ANA.py` would NOT work. Next, include the three lines of comments at the beginning of this file.

The task of this program is to read in two inputs from the user (using the input function):

1. A string (let's call it "a")
2. An integer (let's call it "n")

The program should then output the following pattern:

- The string "a" repeated "n" times
- Followed by the integer "n"
- Followed by the string "a" repeated "n" times again

Running a correct implementation of this program should look like:

```
Enter a string: I
Enter an integer: 2
II2II
```

The prompt text/flavor text of your `input()` function does not have to exactly match the example above. The autograder does not check it, so you can use any reasonable prompt text as you want. After finishing the program, run it a few times, try different inputs, and verify the outputs are correct.

3. Implement `divide_apples.py` (5 points)

Create a new program, name it `divide_apples.py`. Next, include the three lines of comments at the beginning of this file. The task of this program is to read in two inputs from the user, store each in a separate variable:

1. The total number of apples (a positive integer)
2. The number of baskets (a positive integer)

The program should then:

1. Calculate how many apples each basket will contain using integer division (`//`).
2. Determine how many apples will be left over using the modulo operator (`%`).
3. Output the result in exactly 4 lines.
    - Line 1: The original number of apples.
    - Line 2: The original number of baskets.
    - Line 3: The number of apples per basket.
    - Line 4: The number of leftover apples.

You may add other strings to make the flow of your output better. Take a look at the examples.

Hint: for this program, you will need to use the floor division operator `//` and the modulo `%` operator. First, think about how to solve this problem by hand; next, write down code to implement your solution. Then test your program by running it with a few example inputs like those listed below.

Running a correct implementation of this program may look like:

```
Enter total apples: 95
Enter number of baskets: 12
95 apples for
12 baskets can be divided as:
7 apples per basket, and
11 leftover apples.

Another example:

Enter total apples: 50
Enter number of baskets: 8
50 apples for
8 baskets can be divided as:
6 apples per basket, and
2 leftover apples.
```

Output requirement: There should be exactly 4 lines of output. Every line of your output must contain one and only one number, and they must be the number of apples per basket and leftover apples on lines 3 and 4, respectively, and in that order. Other text/formatting does not matter. For example, whether the wordings apples or baskets should be plural or singular does not matter. The autograder only checks the correctness of the numbers and ignores other texts.

4. Submit the three `.py` files to Gradescope

Log in to Gradescope, select **Lab 01: Variables and Expressions (code)**, and submit **all of** `echo.py`, `ana.py`, and `divide_apples.py` there **at once**. You can do so in several ways:

- Select all three files, and drag and drop them into the submission area when prompted.
- Or, browse your files when prompted by Gradescope and select all files you want to submit.
- Alternatively, compress all the files you want to submit into a single zip file (the zip file name does not matter); then submit that zip file via drag-and-drop or browsing.
- You can also compress the folder that contains all the files you want to submit as a single zip file (the zip file name does not matter), and upload that zip file via drag-and-drop or browsing.

If you don't submit all required files at once, you won't get credit for the missing files, but you do still get partial credit for the ones you submitted.
If you run into any trouble submitting, Gradescope has a guide on submitting assignments like this.
Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline.

5. Fill out the Lab 1 Questions on Gradescope

Go back to the Gradescope assignments page, select the assignment **Lab 01: Questions**, and fill in your responses there. This is a necessary part of the lab, and it can be edited as many times as you want before the deadline.
If you see a question you do not know how to answer, ask TAs/UCAs in your lab, use Piazza, or go to one of our office hours.

Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.
You may not use any code that is posted on the internet. If you are not sure, it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions, which will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the UMass Academic Honesty Policy and Procedures so you are aware of what this means.
Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty, you should re-read the course policies. We assume you have read the course policies in detail, and by submitting this project, you have provided your virtual signature in agreement with these policies.